10 Event-driven applications and functions

This chapter covers

- Understanding event-driven architectures
- Using RabbitMQ as a message broker
- Implementing functions with Spring Cloud Function
- Processing events with Spring Cloud Stream
- Producing and consuming events with Spring Cloud Stream

In the previous chapters, we worked on a system of distributed applications that interact according to the request/response pattern, a type of synchronous communication. You saw how to design the interaction both in an imperative and a reactive way. In the first case, processing threads would block, waiting for a response from an I/O operation. In the second case, threads would not wait. A response would be processed by any available thread asynchronously once it was received.

Even if the reactive programming paradigm lets you subscribe to producers and process the incoming data asynchronously, the interaction between the two applications is synchronous. The first application (the client) sends a request to the second one (the server) and expects a response to arrive in a short time. How the client processes the response (imperative or reactive) is an implementation detail that doesn't affect the interaction itself. No matter what, a response is expected to arrive.

Cloud native applications should be loosely coupled. The microservices expert Sam Newman identifies a few different types of coupling, including *implementation*, *deployment*, and *temporal* coupling.[1] Let's consider the Polar Bookshop system we've been working on so far.

We can change the implementation of any of the applications without having to change the others. For example, we can re-implement Catalog Service using the reactive paradigm without affecting Order Service. Using a service interface like a REST API, we hide the implementation details, improving loose coupling. All the applications can be deployed independently. They're not coupled, reducing risks and increasing agility.

However, if you think about how the applications we built so far interact, you'll notice that they need other components of the system to be available. Order Service needs Catalog Service to ensure that a user can order a book successfully. We know that failures happen all the time, so we adopted several strategies to ensure resilience even in the face of adversity, or at least ensuring a graceful degradation of functionality. That's a consequence of *temporal coupling*: Order Service and Catalog Service need to be available at the same time to fulfill the system requirements.

Event-driven architectures describe distributed systems that interact by *producing* and *consuming* events. The interaction is asynchronous, solving the problem of temporal coupling. This chapter will cover the basics of event-driven architectures and event brokers. You'll then learn how to implement business logic using the functional programming paradigm and Spring Cloud Function. Finally, you'll use Spring Cloud Stream to expose the functions as message channels via RabbitMQ, building event-driven applications through the publisher/subscriber (pub/sub) model.

NOTE The source code for the examples in this chapter is available in the Chapter10/10-begin, Chapter10/10-intermediate, and Chapter10/10-end folders, containing the initial, intermediate, and final state of the project (**https://github.com/ThomasVitale/cloud-native-spring-in-action**).

10.1 Event-driven architectures

An event is an occurrence. It's something relevant that happened in a system, like a state change, and there can be many sources of events. This chapter will focus on applications, but events can very well be happening in IoT devices, sensors, or networks. When an event occurs, interested parties can be notified. Event notification is usually done through messages, which are data representations of events.

In an event-driven architecture, we identify *event producers* and *event consumers*. A producer is a component that detects the event and sends a notification. A consumer is a component that is notified when a specific event occurs. Producers and consumers don't know each other and work independently. A producer sends an event notification by publishing a message to a channel operated by an event broker that's responsible for collecting and routing messages to consumers. A consumer is notified by the broker when an event occurs and can act upon it.

Producers and consumers have minimal coupling when using a broker that takes the processing and distribution of events on itself. In particular, they are temporally decoupled,

because the interaction is asynchronous. Consumers can fetch and process messages at any time without affecting the producers whatsoever.

In this section, you'll learn the fundamentals of event-driven models and how they can help build more resilient and loosely coupled applications in the cloud.

10.1.1 Understanding the event-driven models

Event-driven architectures can be based on two main models:

- *Publisher/subscriber (pub/sub)*—This model is based on subscriptions. Producers publish events that are sent to all subscribers to be consumed. Events cannot be replayed after being received, so new consumers joining will not be able to get the past events.
- *Event streaming*—In this model, events are written to a log. Producers publish events as they occur, and they are all stored in an ordered fashion. Consumers don't subscribe to them, but they can read from any part of the event stream. In this model, events can be replayed. Clients can join at any time and receive all the past events.

In a basic scenario, consumers receive and process events as they arrive. For specific use cases like pattern matching, they can also process a series of events over a time window. In the event streaming model, consumers have the additional possibility of processing event streams. At the core of event-driven architectures are platforms that can process and route events. For example, RabbitMQ is a common choice to use with the pub/sub model. Apache Kafka is a powerful platform for event stream processing.

The event streaming model is fascinating and growing in popularity, thanks to the many technologies developed in the last few years, allowing you to build real-time data pipelines. It's a complex model, though, that deserves its own book to be taught effectively. In this chapter, I will cover the pub/sub model.

Before we analyze this model in more detail, I'll define some requirements for the Polar Bookshop system, and we'll use them as a means to explore event-driven architectures with the pub/sub model.

10.1.2 Using the pub/sub model

In the Polar Bookshop system, we need to implement an event-driven solution to allow different applications to communicate with each other asynchronously while reducing their coupling. These are the requirements:

- When an order is accepted:
  - Order Service should notify interested consumers of the event.
  - Dispatcher Service should execute some logic to dispatch the order.
- When an order is dispatched:
  - Dispatcher Service should notify consumers interested in such an event.
  - Order Service should update the order status in the database.

If you paid attention, you probably noticed that the requirements don't specify which applications Order Service should notify upon order creation. In our example, only the new Dispatcher Service application will be interested in those events. Still, more applications might subscribe to the order creation events in the future. The beauty of this design is that you can evolve a software system and add more applications without affecting the existing ones at all. For example, you could add a Mail Service that sends an email to users whenever an order they made has been accepted, and Order Service wouldn't even be aware of it.

This type of interaction should be asynchronous and can be modeled with the pub/sub model. Figure 10.1 illustrates the interaction and describes three flows for accepting, dispatching, and updating an order. They are temporally decoupled and executed asynchronously. You will probably notice that the operations for persisting data into a database and for producing events have the same numbered step. That's because they belong to the same unit of work (a *transaction*), as I'll explain later in the chapter.

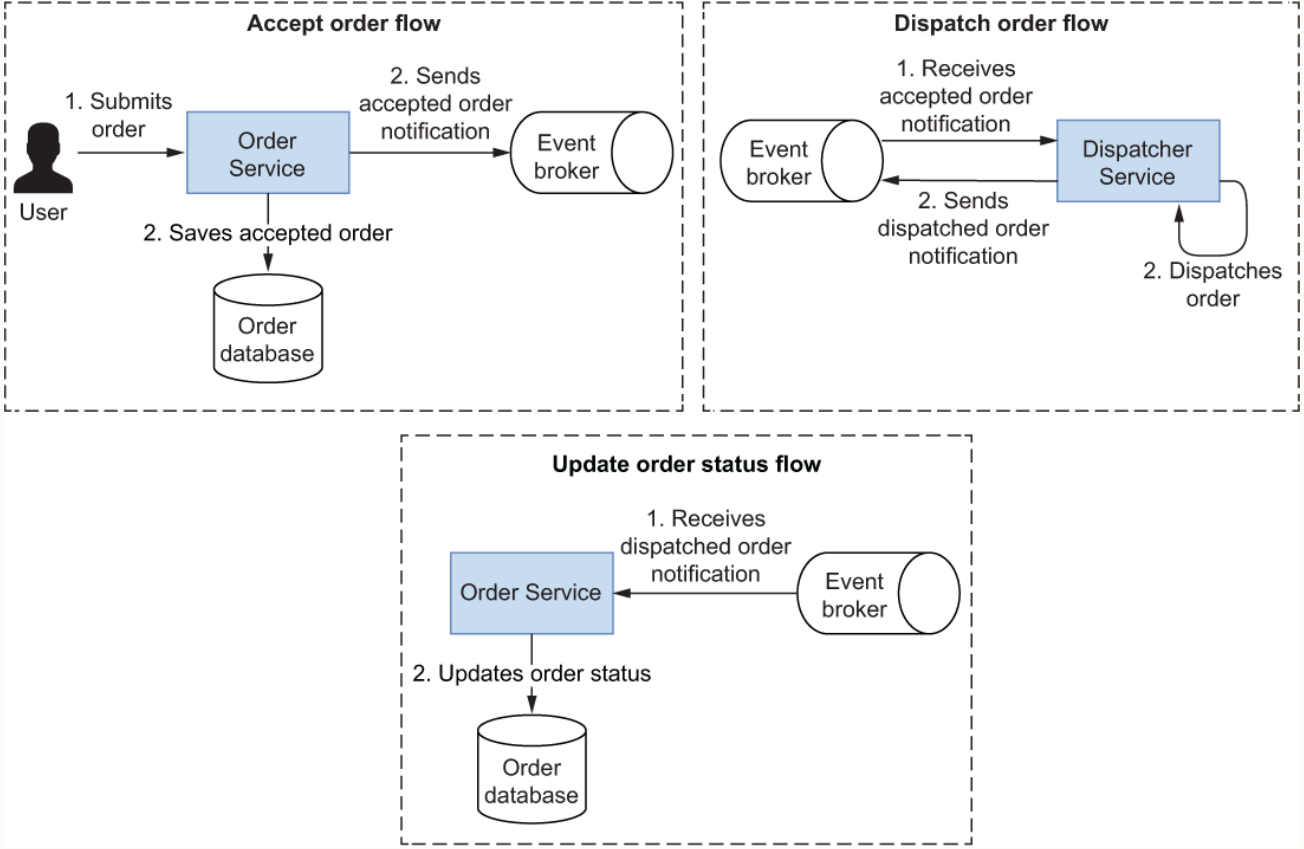


Figure 10.1 Order Service and Dispatcher Service communicate asynchronously and indirectly by producing and consuming events that are collected and distributed by an event broker (RabbitMQ).

In the rest of the chapter, you'll learn a few technologies and patterns you can use to implement this event-driven design for Polar Bookshop. RabbitMQ will be the event-processing platform responsible for collecting, routing, and distributing messages to consumers. Figure 10.2 highlights the event-driven part of the Polar Bookshop system after we introduce the Dispatcher Service application and RabbitMQ.

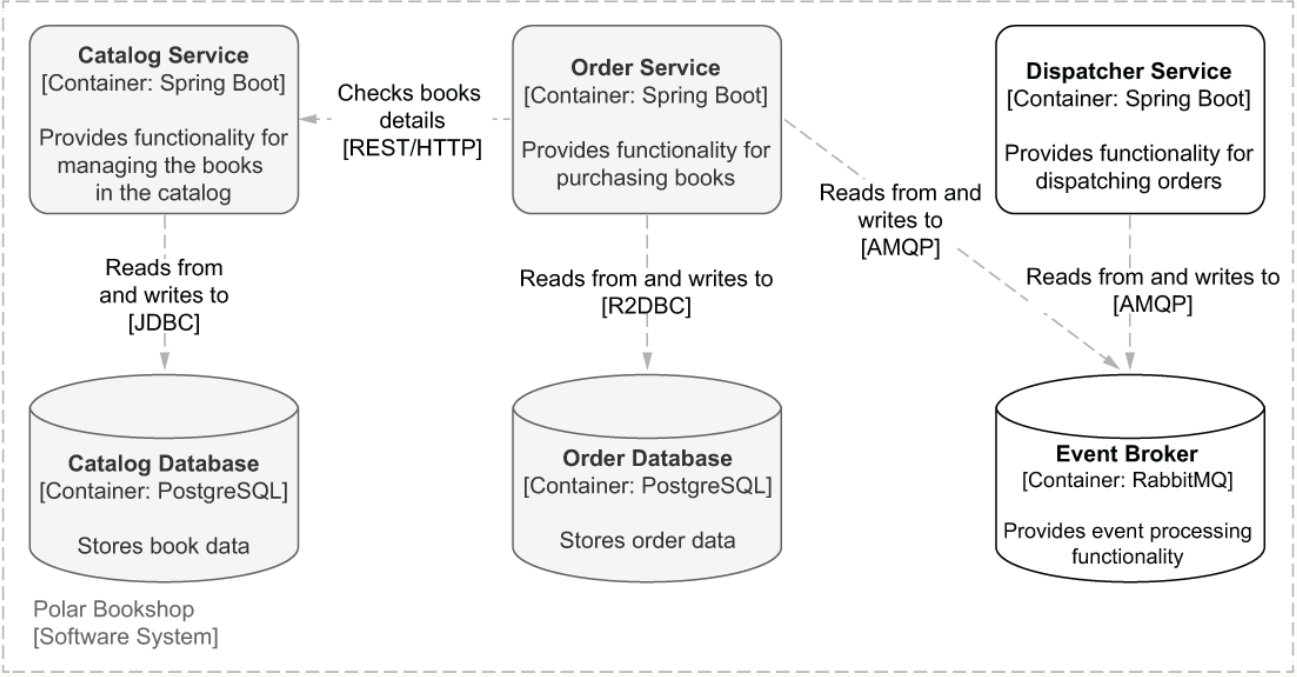


Figure 10.2 In the Polar Bookshop system, Order Service and Dispatcher Service communicate asynchronously based on events distributed by RabbitMQ.

The following section will introduce the basic concepts of RabbitMQ, its protocol, and how to run it in your local environment.

10.2 Message brokers with RabbitMQ

A messaging system requires two main things: a message broker and a protocol. The Advanced Message Queuing Protocol (AMQP) ensures interoperability across platforms and reliable message delivery. It has become widely used in modern architectures, and it's a good fit in the cloud, where we need resilience, loose coupling, and scalability. RabbitMQ is a popular open source message broker that relies on AMQP and provides flexible asynchronous messaging, distributed deployment, and monitoring. Recent RabbitMQ versions have also introduced event streaming features.

Spring provides broad support for the most-used messaging solutions. The Spring Framework itself has built-in support for the Java Message Service (JMS) API. The Spring AMQP project (https://spring.io/projects/spring-amqp) adds support for this messaging protocol and provides integration with RabbitMQ. Apache Kafka is another technology that has become increasingly used in the last few years, such as to implement the event sourcing pattern or real-time stream processing. The Spring for Apache Kafka project (https://spring.io/projects/spring-kafka) provides that integration.

This section will cover the fundamental aspects of the AMQP protocol and RabbitMQ, which is the message broker we'll use to implement messaging in the Polar Bookshop

system. On the application side, we'll use Spring Cloud Stream, which offers convenient and robust integration with RabbitMQ by relying on the Spring AMQP project.

10.2.1 Understanding AMQP for messaging systems

When using an AMQP-based solution like RabbitMQ, the actors involved in the interaction can be categorized as follows:

- *Producer*—The entity sending messages (publisher)
- *Consumer*—The entity receiving messages (subscriber)
- *Message broker*—The middleware accepting messages from producers and routing them to consumers

Figure 10.3 illustrates the interaction between the actors. From the protocol point of view, we can also say that the broker is the *server*, while producers and consumers are the *clients*.

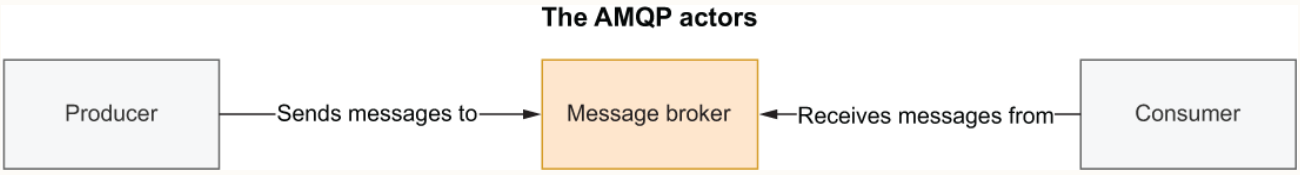


Figure 10.3 In AMQP, a broker accepts messages from producers and routes them to consumers.

NOTE RabbitMQ was initially developed to support AMQP, but it also supports other protocols, including STOMP, MQTT, and even WebSockets for delivering messages over HTTP. Since version 3.9, it also supports event streaming.

The AMQP messaging model is based on *exchanges* and *queues*, as illustrated in figure 10.4. Producers send messages to an exchange. RabbitMQ computes which queues should receive a copy of the message according to a given routing rule. Consumers read messages from a queue.

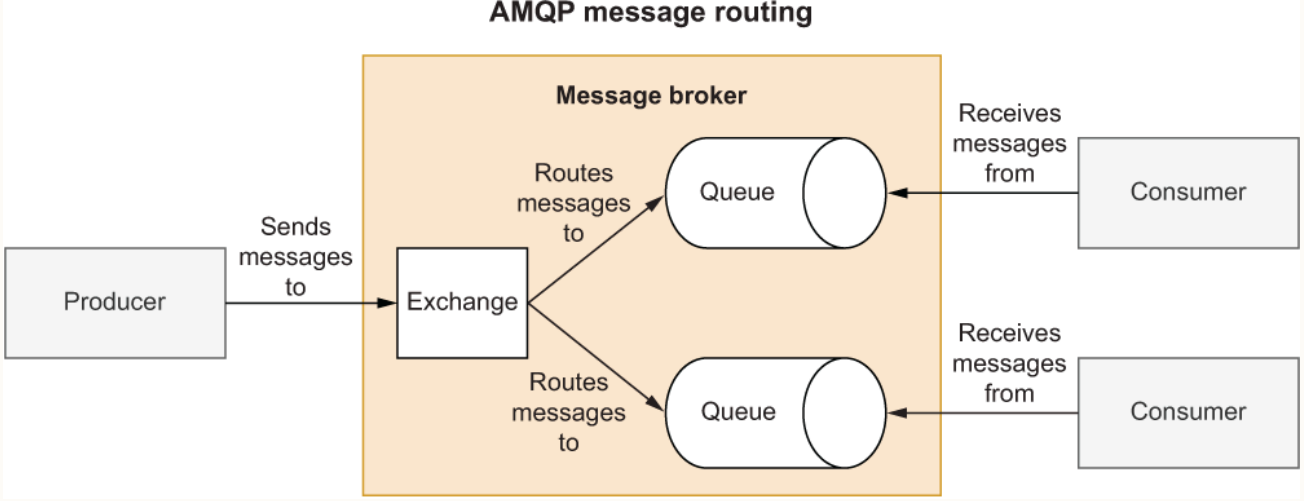


Figure 10.4 Producers publish messages to an exchange. Consumers subscribe to queues. Exchanges route messages to queues according to a routing algorithm.

The protocol establishes that a message comprises attributes and a payload, as shown in figure 10.5. AMQP defines some attributes, but you can add your own to pass the information that's needed to route the message correctly. The payload must be of a binary type and has no constraints besides that.

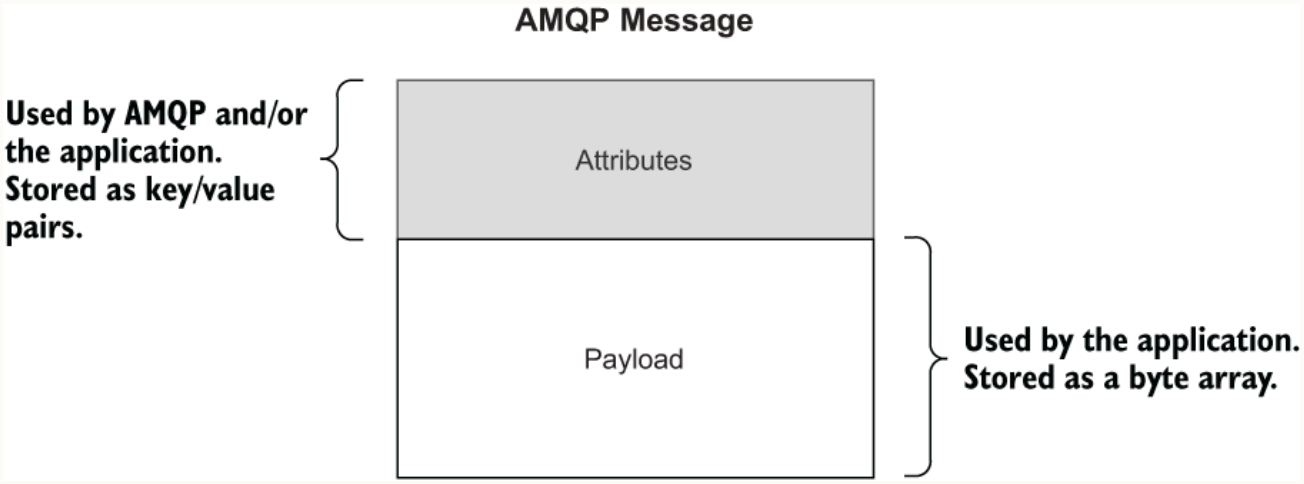


Figure 10.5 An AMQP Message is composed of attributes and a payload.

Now that you know the basics of AMQP, let's get RabbitMQ up and running.

10.2.2 Using RabbitMQ for publish/subscribe communications

RabbitMQ, on top of AMQP, provides a simple yet effective solution for implementing a publish/subscribe interaction that is precisely the one we want to establish between Order Service and Dispatcher Service. Besides the functionality itself, it's essential to look for those properties I addressed in previous chapters for cloud systems and data services, including resilience, high availability, and data replication. RabbitMQ offers all of that. For example, it provides delivery acknowledgment, clustering, monitoring,

queue durability, and replication. Furthermore, several cloud providers offer integrations with managed RabbitMQ services.

For now, you'll run RabbitMQ as a container on your local machine. First, make sure your Docker Engine is running. Then open the docker-compose.yml file located in your polar-deployment repository.

NOTE If you haven't followed along with the examples, you can use Chapter10/10-begin/polar-deployment/docker/docker-compose.yml from the source code accompanying the book as a starting point.

In your docker-compose.yml file, add a new service definition using the RabbitMQ official image (including the management plugin) and expose it through port 5672 (for AMQP) and 15672 (for the management console). The RabbitMQ management plugin is convenient for inspecting exchanges and queues from a browser-based UI.

Listing 10.1 Defining a container for RabbitMQ

```
version: "3.8"
services:
  ...
  polar-rabbitmq:
    image: rabbitmq:3.10-management      (1)
    container_name: polar-rabbitmq
    ports:
      - 5672:5672                        (2)
      - 15672:15672                      (3)
    volumes:                             (4)
      - ./rabbitmq/rabbitmq.conf:/etc/rabbitmq/rabbitmq.conf
```

(1) The official RabbitMQ image with the management plugin enabled

(2) The port where RabbitMQ listens for AMQP requests

(3) The port that exposes the management GUI

(4) Configuration file mounted as a volume

The configuration is based on a file mounted as a volume, similar to how we configured PostgreSQL. Create a docker/rabbitmq folder within your polar-deployment repository, and add a new rabbitmq.conf file to configure the default account.

Listing 10.2 Configuring RabbitMQ default account

```
default_user = user
default_pass = password
```

Next, open a Terminal window, navigate to the folder where your docker-compose.yml file is located, and run the following command to start RabbitMQ:

```
$ docker-compose up -d polar-rabbitmq
```

Finally, open a browser window and navigate to http://localhost:15672 to access the RabbitMQ management console. Log in using the credentials we defined in the configuration file (user/password) and have a look around. In the following sections you'll be able to follow the message flows between Order Service and Dispatcher Service in the Exchanges and Queues areas of the management console.

When you're done exploring the RabbitMQ management console, you can shut it down as follows:

```
$ docker-compose down
```

Spring Cloud Stream helps integrate applications with event brokers like RabbitMQ seamlessly. But before we get to it, we need to define the logic that will process messages. In the next section, you'll learn about Spring Cloud Function and how to implement the business logic of the new order flow in terms of suppliers, functions, and consumers.

10.3 Functions with Spring Cloud Function

Oleg Zhurakousky, project lead for Spring Cloud Function and Spring Cloud Stream, often asks conference audiences this question: Is there any business feature that you cannot define in terms of suppliers, functions, and consumers? It's an interesting and challenging question. Can you think of anything? Most software requirements can be expressed with functions.

Why use functions in the first place? They are a simple, uniform, and portable programming model that is a perfect fit for event-driven architectures, inherently based on these concepts.

Spring Cloud Function promotes the implementation of business logic via functions based on the standard interfaces introduced by Java 8: Supplier, Function, and Consumer.

- *Supplier*—A supplier is a function with only output, no input. It's also known as a *producer*, *publisher*, or *source*.
- *Function*—A function has both input and output. It's also known as a *processor*.
- *Consumer*—A consumer is a function with input but no output. It's also known as a *subscriber* or *sink*.

In this section, you'll learn how Spring Cloud Function works and how to implement business logic via functions.

10.3.1 Using the functional paradigm in Spring Cloud Function

Let's get started with functions by considering the business requirements I listed earlier for the Dispatcher Service application. Whenever an order is accepted, Dispatcher Service should be responsible for packing and labeling the order, and for notifying interested parties (in this case, the Order Service) once the order has been dispatched. For simplicity, let's assume that both the *pack* and *label* actions are performed by the application itself, and we'll consider how to implement the business logic via functions before even thinking about frameworks.

The two actions to be performed as part of dispatching an order could be represented as functions:

- The *pack* function takes the identifier of an accepted order as input, packs the order (in the example, the processing is represented by a log message), and returns the order identifier as output, ready to be labeled.
- The *label* function takes the identifier of a packed order as input, labels the order (in the example, the processing is represented by a log message), and returns the order identifier as output, completing the dispatch.

The composition of these two functions in sequence gives the full implementation of the business logic for Dispatcher Service, as shown in figure 10.6.

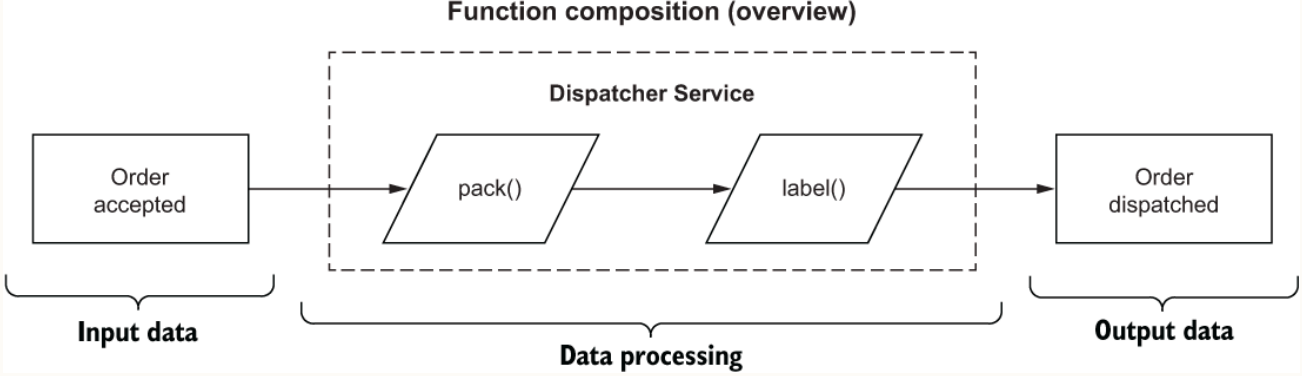


Figure 10.6 The business logic for Dispatcher Service is implemented as a composition of two functions: pack and label.

Let's see how we can implement these functions and what Spring Cloud Function brings to the table.

INITIALIZING A SPRING CLOUD FUNCTION PROJECT

You can initialize the Dispatcher Service project from Spring Initializr (**https://start.spring.io**) and store the result in a new dispatcher-service Git repository. The parameters for the initialization are shown in figure 10.7.

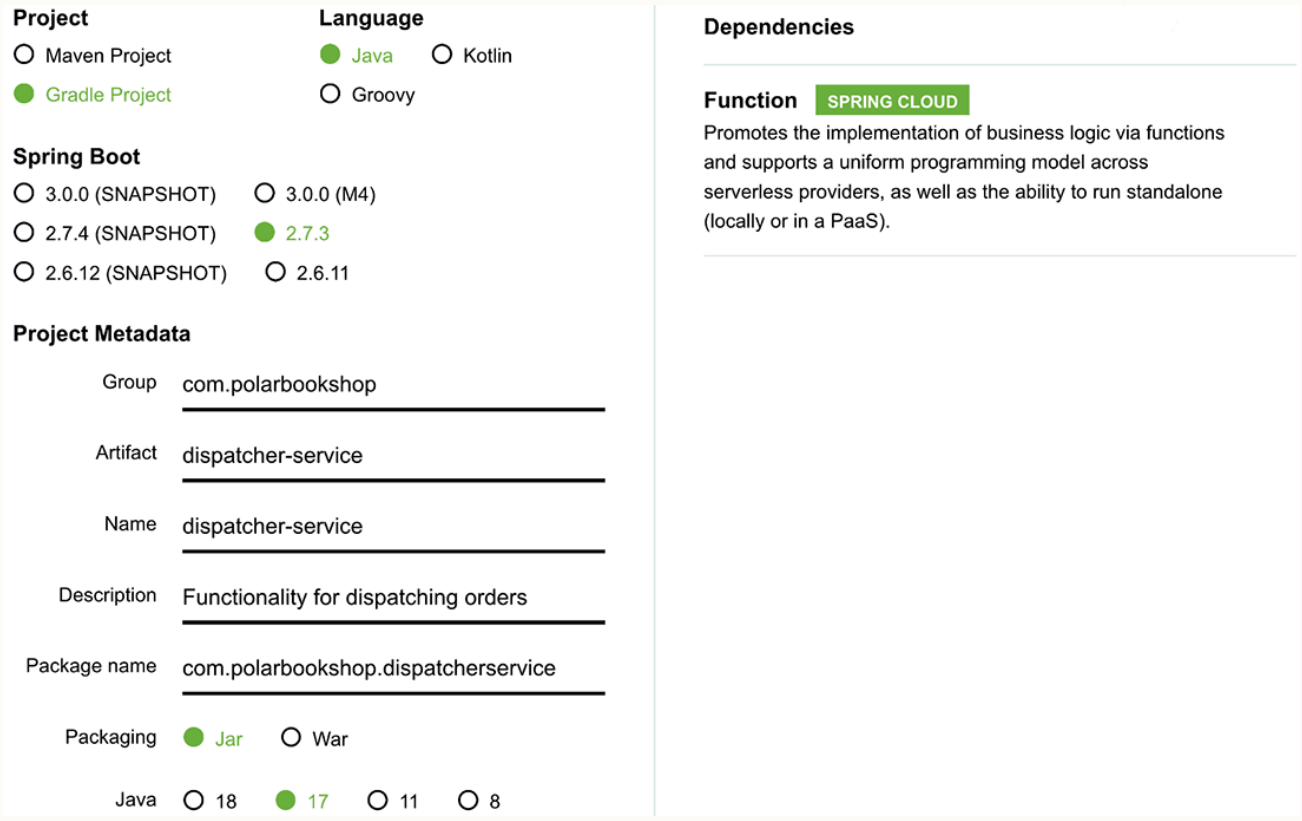


Figure 10.7 The parameters for initializing the Dispatcher Service project

TIP In the begin folder for this chapter, you can find a curl command that you can run in a Terminal window. It downloads a zip file containing all the code you need to get started, without going through the manual generation on the Spring Initializr website.

The resulting dependencies section of the build.gradle file looks like this:

```
dependencies {
  implementation 'org.springframework.boot:spring-boot-starter'
  implementation 'org.springframework.cloud:spring-cloud-function-context'
  testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

The main dependencies are

- *Spring Boot* (org.springframework.boot:spring-boot-starter)—Provides basic Spring Boot libraries and auto-configuration.
- *Spring Cloud Function* (org.springframework.cloud:spring-cloud-function-context)—Provides the Spring Cloud Function libraries that promote and support business logic implementation via functions.
- *Spring Boot Test* (org.springframework.boot:spring-boot-starter-test)—Provides several libraries and utilities for testing applications, including Spring Test, JUnit, AssertJ, and Mockito. It's automatically included in every Spring Boot project.

Next, rename the autogenerated application.properties file to application.yml, and configure the server port and application name. At present, the application doesn't contain a web server. Nevertheless, we'll configure the server port number because it will be used when we add monitoring capabilities to the application in chapter 13.

Listing 10.3 Configuring server and application name

```
server:
  port: 9003                  ❶
spring:
  application:
    name: dispatcher-service  ❷
```

❶ The port that will be used by the embedded web server

❷ The name of the application

Next, let's see how we can implement the business logic using functions.

The business logic can be implemented in a standard way by using the Java `Function` interface. No Spring needed.

Let's first consider the *pack* function. The input of the function should provide the identifier of an order that has previously been accepted. We can model this data via a simple DTO.

In the `com.polarbookshop.dispatcherservice` package, create an `OrderAcceptedMessage` record to hold the order identifier.

Listing 10.4 A DTO representing the event about orders being accepted

```
package com.polarbookshop.dispatcherservice;

public record OrderAcceptedMessage (     ❶
  Long orderId
){}
```

❶ DTO containing the order identifier as a Long field

NOTE Modeling events is a fascinating topic that goes beyond Spring and would require a few chapters to cover properly. If you'd like to learn more about this subject, I recommend reading these articles by Martin Fowler: "Focusing on Events" (**https://martinfowler.com/eaaDev/EventNarrative.html**); "Domain Event" (**https://martinfowler.com/eaaDev/DomainEvent.html**); and "What do you mean by 'Event-Driven'?" (**https://martinfowler.com/articles/ 201701-event-driven.html**), all on his **MartinFowler.com** blog.

The output of the function can be the simple identifier of the packed order represented as a `Long` object.

Now that input and output are clear, it's time to define the function. Create a new `DispatchingFunctions` class, and add a `pack()` method to implement order packing as a function.

Listing 10.5 Implementing the "pack" action as a function

```
package com.polarbookshop.dispatcherservice;

import java.util.function.Function;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class DispatchingFunctions {
  private static final Logger log =
    LoggerFactory.getLogger(DispatchingFunctions.class);

  public Function<OrderAcceptedMessage, Long> pack() {    ❶
    return orderAcceptedMessage -> {                      ❷
      log.info("The order with id {} is packed.",
        orderAcceptedMessage.orderId());
      return orderAcceptedMessage.orderId();              ❸
    };
  }
}
```

❶ Function implementing the order-packing business logic

❷ It takes an OrderAcceptedMessage object as input.

❸ Returns an order identifier (Long) as output

You can see how there's only standard Java code in this listing. I strive to provide real-world examples in this book, so you might wonder what's happening here. In this case, I decided to focus on the essential aspects of using the functional programming paradigm in the context of an event-driven application. Inside the function, you can add any processing logic you like. What matters here is the contract provided by the function, its signature: inputs and outputs. After defining that, you're free to implement the function as needed. I could have provided a more real-world implementation of this function, but it would have added nothing valuable considering the goal of this chapter. It doesn't even have to be Spring-based code. And in this example, it's not: it's plain Java code.

Spring Cloud Function is capable of managing functions defined in different ways, as long as they adhere to the standard Java interfaces `Function`, `Supplier`, and `Consumer`. You can make Spring Cloud Function aware of your functions by registering them as beans. Go ahead and do that for the `pack()` function by annotating the `DispatchingFunctions` class as `@Configuration` and the method as `@Bean`.

Listing 10.6 Configuring the function as a bean

```
@Configuration                                    ❶
public class DispatchingFunctions {
  private static final Logger log =
    LoggerFactory.getLogger(DispatchingFunctions.class);

  @Bean                                           ❷
  public Function<OrderAcceptedMessage, Long> pack() {
    return orderAcceptedMessage -> {
      log.info("The order with id {} is packed.",
        orderAcceptedMessage.orderId());
      return orderAcceptedMessage.orderId();
    };
  }
}
```

❶ Functions are defined in a configuration class.

❷ Functions defined as beans can be discovered and managed by Spring Cloud Function.

As you'll see later, functions registered as beans are enhanced with extra features by the Spring Cloud Function framework. The beauty of this is that the business logic itself is not aware of the surrounding framework. You can evolve it independently and test it without being concerned about framework-related issues.

USING IMPERATIVE AND REACTIVE FUNCTIONS

Spring Cloud Function supports both imperative and reactive code, so you're free to implement functions using reactive APIs like Mono and Flux. You can also mix and match. For the sake of the example, let's implement the *label* function using Project Reactor. The input of the function will be the identifier of an order that has been packed, represented as a Long object. The output of the function will be the identifier of the order that has been labeled, resulting in the dispatching process being complete. We can model such data via a simple DTO, just like we did for OrderAcceptedMessage.

In the com.polarbookshop.dispatcherservice package, create an OrderDispatchedMessage record to hold the identifier for a dispatched order.

Listing 10.7 A DTO representing the event about orders being dispatched

```
package com.polarbookshop.dispatcherservice;

public record OrderDispatchedMessage (      ❶
  Long orderId
){}
```

❶ DTO containing the order identifier as a Long field

Now that the input and output are clear, it's time to define the function. Open the DispatchingFunctions class and add a label() method to implement the order labeling as a function. Since we want it to be reactive, both input and output are wrapped in a Flux publisher.

Listing 10.8 Implementing the "label" action as a function

```
package com.polarbookshop.dispatcherservice;

import java.util.function.Function;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import reactor.core.publisher.Flux;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DispatchingFunctions {
  private static final Logger log =
    LoggerFactory.getLogger(DispatchingFunctions.class);

  ...

  @Bean                                                    ❶
  public Function<Flux<Long>, Flux<OrderDispatchedMessage>> label() {
    return orderFlux -> orderFlux.map(orderId -> {                  ❷
      log.info("The order with id {} is labeled.", orderId);
      return new OrderDispatchedMessage(orderId);                   ❸
    });
  }
}
```

❶ Function implementing the order-labeling business logic

❷ It takes an order identifier (Long) as input.

❸ Returns an OrderDispatchedMessage as output

We have just implemented both functions, so let's see how we can combine and use them.

10.3.2 Composing and integrating functions: REST, serverless, data streams

The implementation of the business logic for Dispatcher Service is almost done. We still need a way to compose the two functions. Based upon our requirements, dispatching an order consists of two steps to be executed in sequence: pack() first and label() after.

Java provides features to compose Function objects in sequence using the andThen() or compose() operators. The problem is that you can use them only when the output type of the first function is the same as the second function's input. Spring Cloud Function provides a solution to that problem and lets you compose functions seamlessly through transparent type conversion, even between imperative and reactive functions like those we defined earlier.

Composing functions with Spring Cloud is as simple as defining a property in your application.yml (or application.properties) file. Open the application.yml file in your Dispatcher Service project, and configure Spring Cloud Function to manage and compose the pack() and label() functions as follows.

Listing 10.9 Declaring the functions managed by Spring Cloud

```
spring:
  cloud:
    function:
      definition: pack|label    ❶
```

❶ Definition of the function managed by Spring Cloud Function

The spring.cloud.function.definition property lets you declare which functions you want Spring Cloud Function to manage and integrate, resulting in a specific data flow. In the previous section, we implemented the basic pack() and label() functions. Now we can instruct Spring Cloud Function to use them as building blocks and produce a new function that comes from the composition of those two.

In a serverless application like those meant to be deployed on a FaaS platform (such as AWS Lambda, Azure Functions, Google Cloud Functions, or Knative), you would usually have one function defined per application. The cloud function definition can be mapped one-to-one to a function declared in your application, or you can use the *pipe* (|) operator to compose functions together in a data flow. If you need to define multiple functions, you can use the semicolon (;) character as the separator instead of the pipe (|).

To sum up, you only need to implement standard Java functions, and you can then configure Spring Cloud Function to use them as they are or after combining them. The framework will do the rest, including transparently converting input and output types to make the composition possible. Figure 10.8 illustrates the function composition.

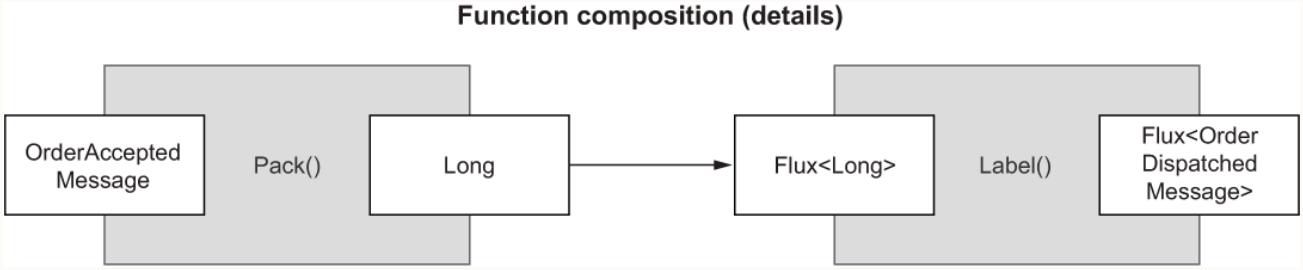


Figure 10.8 You can combine functions with different input and output types and mix imperative and reactive types as well. Spring Cloud Function will transparently handle any type conversion.

At this point you're probably wondering how you can use these functions. That's my favorite part. Once you define the functions, the framework can expose them in different ways depending on your needs. For example, Spring Cloud Function can automatically expose the functions defined in `spring.cloud.function.definition` as REST endpoints. Then you can directly package the application, deploy it on a FaaS platform like Knative, and voilà: you've got your first serverless Spring Boot application. That's what we'll do in chapter 16 when we build serverless applications. Or you can use one of the adapters provided by the framework to package the application and deploy it on AWS Lambda, Azure Functions, or Google Cloud Functions. Or you can combine it with Spring Cloud Stream and bind the function to message channels in an event broker like RabbitMQ or Kafka.

Before we explore the integration with RabbitMQ using Spring Cloud Stream, I want to show you how to test the functions and their composition in isolation. Once the business logic is implemented as functions and tested, we can be sure it will work the same way, whether it's triggered by a REST endpoint or an event notification.

10.3.3 Writing integration tests with @FunctionalSpringBootTest

Using the functional programming paradigm, we can implement business logic in standard Java and write unit tests with JUnit without being affected by the framework. At that level there is no Spring code, just plain Java. Once you've ensured that each function works, you'll want to write some integration tests to verify your application's overall behavior when your functions are processed by Spring Cloud Function and exposed the way you configured.

Spring Cloud Function provides a `@FunctionalSpringBootTest` annotation you can use to set up the context for your integration tests. Unlike unit tests, you don't want to invoke the function directly but rather ask the framework to provide that for you. All the functions managed by the framework are available through the `FunctionCatalog`, an object that acts as a function registry. When the framework serves the function, it doesn't only contain the implementation you wrote; it's enhanced with extra features offered by Spring Cloud Function, like transparent type conversion and function composition. Let's see how this works.

First, you need to add a test dependency on Reactor Test in the build.gradle file, since part of the business logic is implemented using Reactor. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 10.10 Adding dependency for Reactor Test in Dispatcher Service

```
dependencies {
  ...
  testImplementation 'io.projectreactor:reactor-test'
}
```

Then, in the src/test/java folder of your Dispatcher Service project, create a new `DispatchingFunctionsIntegrationTests` class. You can write integration tests for the two functions individually, but it's more interesting verifying the behavior of the composed function, `pack() + label()`, as provided by Spring Cloud Function.

Listing 10.11 Integration tests for a function composition

```
package com.polarbookshop.dispatcherservice;

import java.util.function.Function;
import org.junit.jupiter.api.Test;
```

```
import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cloud.function.context.FunctionCatalog;
import org.springframework.cloud.function.context.test
➥.FunctionalSpringBootTest;

@FunctionalSpringBootTest
class DispatchingFunctionsIntegrationTests {

  @Autowired
  private FunctionCatalog catalog;

  @Test
  void packAndLabelOrder() {
    Function<OrderAcceptedMessage, Flux<OrderDispatchedMessage>>
      packAndLabel = catalog.lookup(
       Function.class,
        "pack|label");                    ❶
    long orderId = 121;

    StepVerifier.create(packAndLabel.apply(
      new OrderAcceptedMessage(orderId)       ❷
     ))
     .expectNextMatches(dispatchedOrder ->     ❸
       dispatchedOrder.equals(new OrderDispatchedMessage(orderId)))
     .verifyComplete();
  }
}
```

❶ Gets the composed function from the FunctionCatalog

❷ Defines an OrderAccepted-Message, which is the input to the function

❸ Asserts that the output of the function is the expected OrderDispatchedMessage object

Finally, open a Terminal window, navigate to the Dispatcher Service project root folder, and run the tests:

```
$ ./gradlew test --tests DispatchingFunctionsIntegrationTests
```

This type of integration test ensures the correct behavior of the defined cloud function, independently of how it will be exposed. In the source code accompanying the book, you'll find a broader set of autotests (Chapter10/10-intermediate/dispatcher-service).

Functions are a simple yet effective way to implement business logic and delegate infrastructural concerns to the framework. In the next section you'll learn how to bind functions to message channels on RabbitMQ using Spring Cloud Stream.

10.4 Processing messages with Spring Cloud Stream

The principles that drive the Spring Cloud Function framework can also be found in Spring Cloud Stream. The idea is that you, as a developer, are responsible for the business logic, while the framework handles infrastructural concerns like how to integrate a message broker.

Spring Cloud Stream is a framework for building scalable, event-driven, and streaming applications. It's built on top of Spring Integration, which offers the communication layer with message brokers; Spring Boot, which provides auto-configuration for the middleware integration; and Spring Cloud Function, which produces, processes, and consumes events. Spring Cloud Stream relies on the native features of each message broker, but it also provides an abstraction to ensure a seamless experience independently of the underlying middleware. For example, features like consumer groups and partitions (native in Apache Kafka) are not present in RabbitMQ, but you can still use them thanks to the framework providing them for you.

My favorite Spring Cloud Stream feature is that you can drop a dependency in a project like Dispatcher Service and get functions automatically bound to an external message broker. The best part of it? You don't have to change any code in the application, just the configuration in application.yml or application.properties. In previous versions of the framework, it was necessary to use dedicated annotations to match the business logic with the Spring Cloud Stream components. Now it's completely transparent.

The framework supports integrations with RabbitMQ, Apache Kafka, Kafka Streams, and Amazon Kinesis. There are also integrations maintained by partners for Google PubSub, Solace PubSub+, Azure Event Hubs, and Apache RocketMQ.

This section will cover how to expose the composed function we defined in Dispatcher Service through message channels in RabbitMQ.

10.4.1 Configuring the integration with RabbitMQ

Spring Cloud Stream is based on a few essential concepts:

- *Destination binder*—The component providing the integration with external messaging systems, like RabbitMQ or Kafka
- *Destination binding*—The bridge between the external messaging system entities, like queues and topics, and the application-provided producers and consumers
- *Message*—The data structure used by the application producers and consumers to communicate with the destination binders, and therefore with the external messaging systems

All three of these are handled by the framework itself. The core of your application, the business logic, is not aware of the external messaging system. Destination binders are responsible for letting the application communicate with the external message brokers, including any vendor-specific concerns. The bindings are auto-configured by the framework, but you can still provide your own configuration to adapt them to your needs, as we'll do for Dispatcher Service. Figure 10.9 shows a model of a Spring Boot application using Spring Cloud Stream.

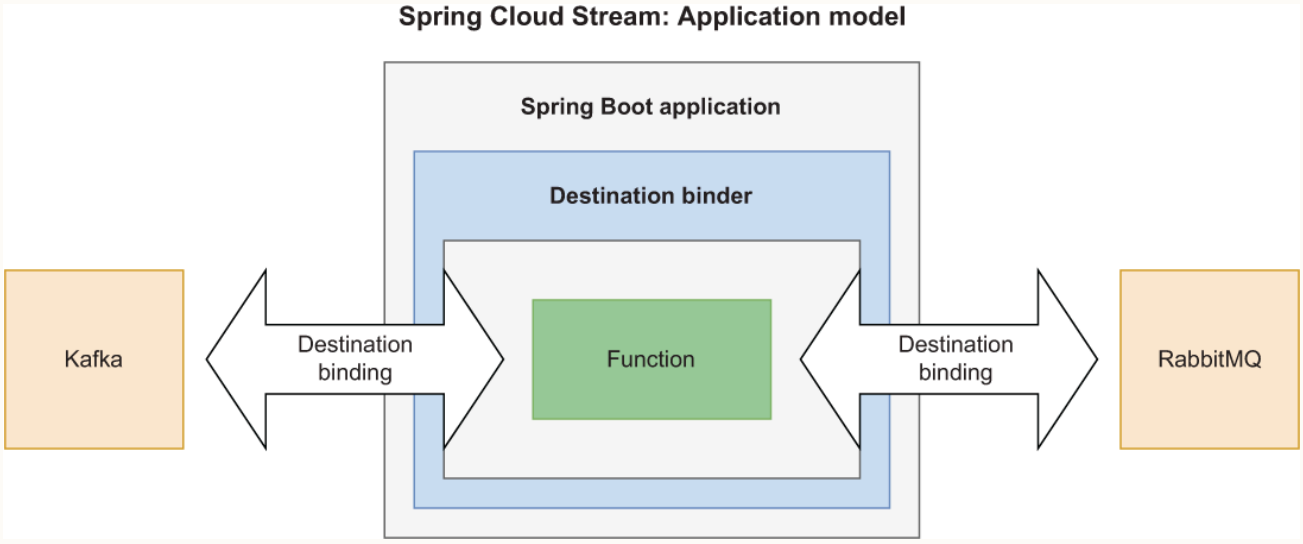


Figure 10.9 In Spring Cloud Stream, a destination binder provides integration with external messaging systems and establishes message channels with them.

Once you have defined the business logic of your application as functions, and you've configured Spring Cloud Function to manage them (like we did for Dispatcher Service), you can expose the functions through a message broker by adding a dependency on the Spring Cloud Stream binder project specific to the broker you want to use. I'll show you how to work with RabbitMQ both for input and output message channels, but you can also bind to multiple messaging systems within the same application.

First, open the build.gradle file for the Dispatcher Service project (dispatcher-service), and replace the Spring Cloud Function dependency with the RabbitMQ binder for Spring Cloud Stream. Since Spring Cloud Function is already included in Spring Cloud Stream, you don't need to add it explicitly. You can also remove the dependency on Spring Boot Starter, which is also included in the Spring Cloud Stream dependency. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 10.12 Updating dependencies in Dispatcher Service

```
dependencies {
  implementation
  ➥'org.springframework.cloud:spring-cloud-stream-binder-rabbit'
  testImplementation 'org.springframework.boot:spring-boot-starter-test'
  testImplementation 'io.projectreactor:reactor-test'
}
```

Next, open the application.yml file and add the following configuration for the RabbitMQ integration. Port, username, and password are the same ones we previously defined in Docker Compose (listings 10.1 and 10.2).

Listing 10.13 Configuring the RabbitMQ integration

```
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: user
    password: password
    connection-timeout: 5s
```

That's it. If you run the Dispatcher Service, you'll notice it already works perfectly without further configuration. Spring Cloud Stream will auto-generate and configure the bindings to exchanges and queues in RabbitMQ.

That's great for getting up and running quickly, but you will probably want to add your own configuration to customize the behavior for a production scenario. The following section will show you how to do that, again without changing any code in your business logic. How great is that?

10.4.2 Binding functions to message channels

Getting started with Spring Cloud Stream is straightforward, but there's a chance of confusing concepts with similar names. In the context of message brokers and Spring Cloud Stream, the term *binding* and its variations are used a lot and can lead to misunderstandings. Figure 10.10 shows all the entities in place.

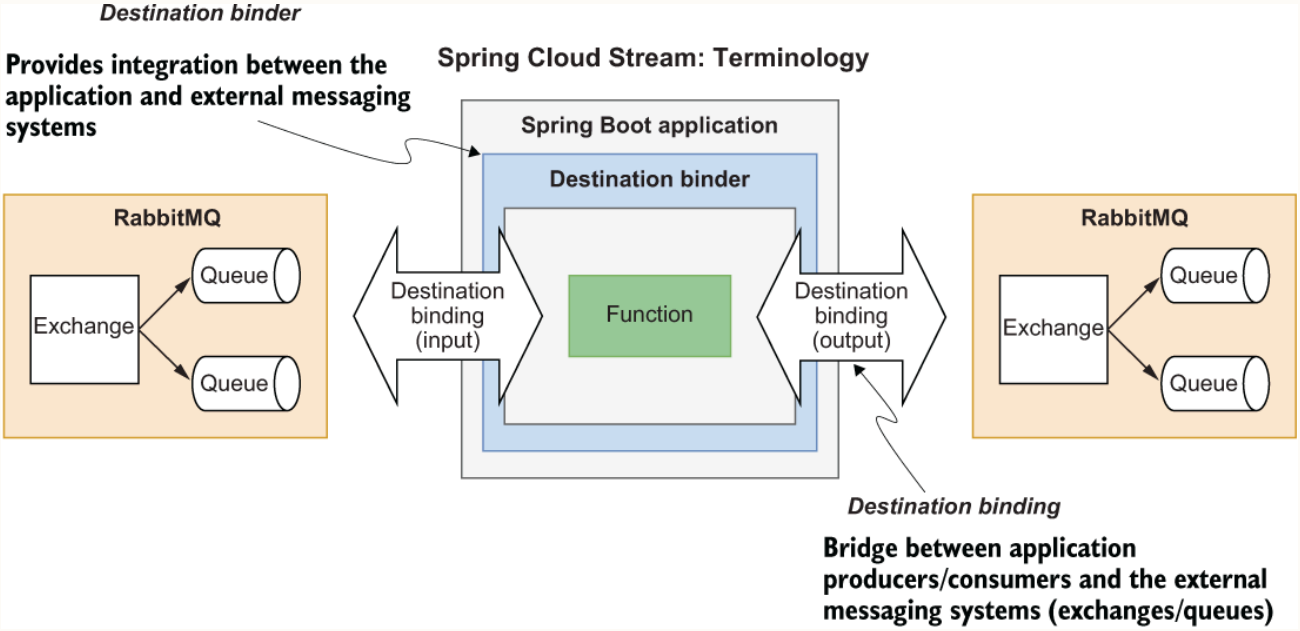


Figure 10.10 In Spring Cloud Stream, bindings establish message channels between applications and message brokers.

Spring Cloud Stream provides a Spring Boot application with a *destination binder* that integrates with an external messaging system. The binder is also responsible for establishing communication channels between the application producers and consumers and the messaging system entities (exchanges and queues for RabbitMQ). These communication channels are called *destination bindings*, and they are bridges between applications and brokers.

A *destination binding* can be either an input channel or an output channel. By default, Spring Cloud Stream maps each binding (both input and output) to an exchange in RabbitMQ (a *topic exchange*, to be more precise). Furthermore, for each input binding, it binds a queue to the related exchange. That's the queue from which consumers receive and process events. This setup provides all the plumbing for implementing event-driven architectures based on the pub/sub model.

In the following sections, I'll tell you more about destination bindings in Spring Cloud Stream and how they relate to exchanges and queues in RabbitMQ.

UNDERSTANDING DESTINATION BINDINGS

As you can see in figure 10.10, destination bindings are an abstraction representing a bridge between application and broker. When using the functional programming model, Spring Cloud Stream generates an input binding for each function accepting input data, and an output binding for each function returning output data. Each binding is assigned a logical name following this convention:

- Input binding: <functionName> + -in- + <index>
- Output binding: <functionName> + -out- + <index>

Unless you use partitions (for example, with Kafka), the <index> part of the name will always be 0. The <functionName> is computed from the value of the spring.cloud.function.definition property. In case of a single function, there is a one-to-one mapping. For example, if in Dispatcher Service we only had one function called dispatch, the related binding would be named dispatch-in-0 and dispatch-out-0. We actually used a composed function (pack|label), so the binding names are generated by combining the names of all the functions involved in the composition:

- Input binding: packlabel-in-0
- Output binding: packlabel-out-0

These names are only relevant for configuring the bindings themselves in the application. They're like unique identifiers that let you reference a specific binding and apply custom configuration. Notice that these names exist only in Spring Cloud Stream—they're logical names. RabbitMQ doesn't know about them.

CONFIGURING DESTINATION BINDINGS

By default, Spring Cloud Stream uses the binding names to generate the names for exchanges and queues in RabbitMQ, but in a production scenario you'd probably want to manage them explicitly for several reasons. For example, it's likely that both exchanges and queues already exist in production. You will also want to control different options for exchanges and queues, like durability or routing algorithms.

For Dispatcher Service, I'll show you how to configure input and output bindings. At startup, Spring Cloud Stream will check if the related exchanges and queues already exist in RabbitMQ. If they don't, it will create them according to your configuration.

Let's start by defining the destination names that will be used to name exchanges and queues in RabbitMQ. In your Dispatcher Service project, update the application.yml file as follows.

Listing 10.14 Configuring Cloud Stream bindings and RabbitMQ destinations

```
spring:
  cloud:
    function:
      definition: pack|label
    stream:
      bindings:                          (1)
        packlabel-in-0:                  (2)
          destination: order-accepted    (3)
          group: ${spring.application.name}   (4)
        packlabel-out-0:                 (5)
          destination: order-dispatched  (6)
```

(1) Section for configuring destination bindings

(2) The input binding

(3) The actual name at the broker that the binder binds to (the exchange in RabbitMQ)

(4) The consumer group interested in the destination (same as the application name)

(5) The output binding

(6) The actual name at the broker that the binder binds to (the exchange in RabbitMQ)

The output binding (packlabel-out-0) will be mapped to an order-dispatched exchange in RabbitMQ. The input binding (packlabel-in-0) will be mapped to an order-accepted exchange and an order-accepted.dispatcher-service queue in RabbitMQ. If they don't exist already in RabbitMQ, the binder will create them. The queue-naming strategy (<destination>.<group>) includes a parameter called *consumer group*.

The idea of *consumer groups* has been borrowed from Kafka and is very useful. In a standard pub/sub model, all consumers receive a copy of the messages sent to the queues they're subscribed to. That is convenient when different applications need to process the messages. But in a cloud native context, where multiple instances of an application are

running simultaneously for scaling and resilience, that would be a problem. If you have numerous Dispatcher Service instances, you don't want an order to be dispatched from all of them. That would lead to errors and an inconsistent state.

Consumer groups solve the problem. All consumers in the same group share a single subscription. As a consequence, each message arriving at the queue to which they're subscribed will be processed by one consumer only. Assume we have two applications (Dispatcher Service and Mail Service) interested in receiving events about accepted orders and deployed in a replicated fashion. Using the application name to configure consumer groups, we can ensure that each event is received and processed by a single instance of Dispatcher Service and a single instance of Mail Service, as shown in figure 10.11.

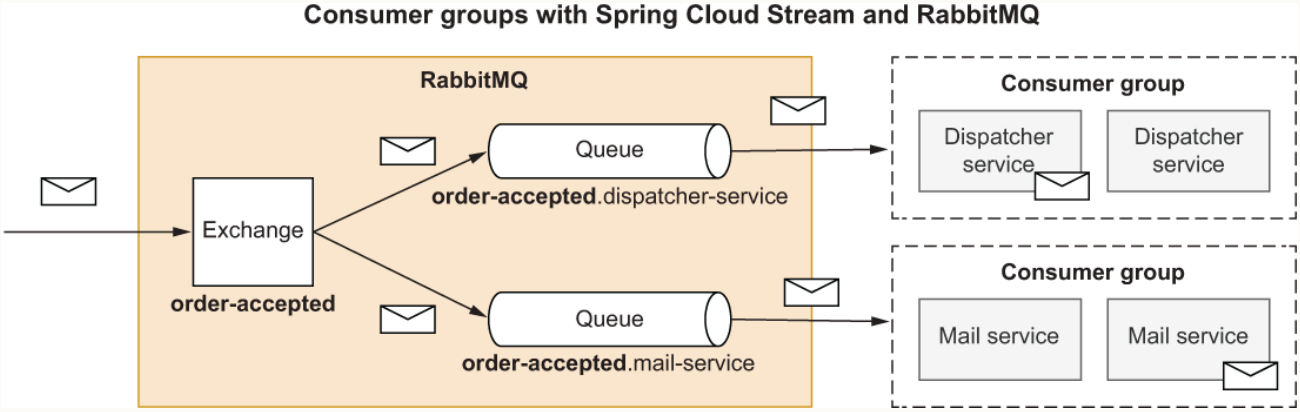


Figure 10.11 Consumer groups ensure that each message is received and processed by only one consumer within the same group.

EXPLORING EXCHANGES AND QUEUES IN RABBITMQ

After configuring the integration with RabbitMQ through Spring Cloud Stream, it's time to try running Dispatcher Service.

First, start a RabbitMQ container. Open a Terminal window, navigate to the folder in your polar-deployment repository where you keep your docker-compose.yml file (polar-deployment/docker), and run the following command:

```
$ docker-compose up -d polar-rabbitmq
```

Then open another Terminal window, navigate to the Dispatcher Service project's root folder (dispatcher-service), and run the application as follows:

```
$ ./gradlew bootRun
```

The application logs will already give you a hint of what happened, but for a clearer understanding, let's check the RabbitMQ management console (exposed through port 15672).

Open a browser window and navigate to http://localhost:15672. The credentials are the same that we defined in Docker Compose (user/password). Then go to the Exchanges section. Figure 10.12 shows a list of default exchanges provided by RabbitMQ and the two exchanges generated by our application: order-accepted and order-dispatched. Spring Cloud Stream maps them to the packlabel-in-0 and packlabel-out-0 bindings respectively. The exchanges are *durable* (denoted by the D icon in the management console), meaning that they will survive a broker restart.

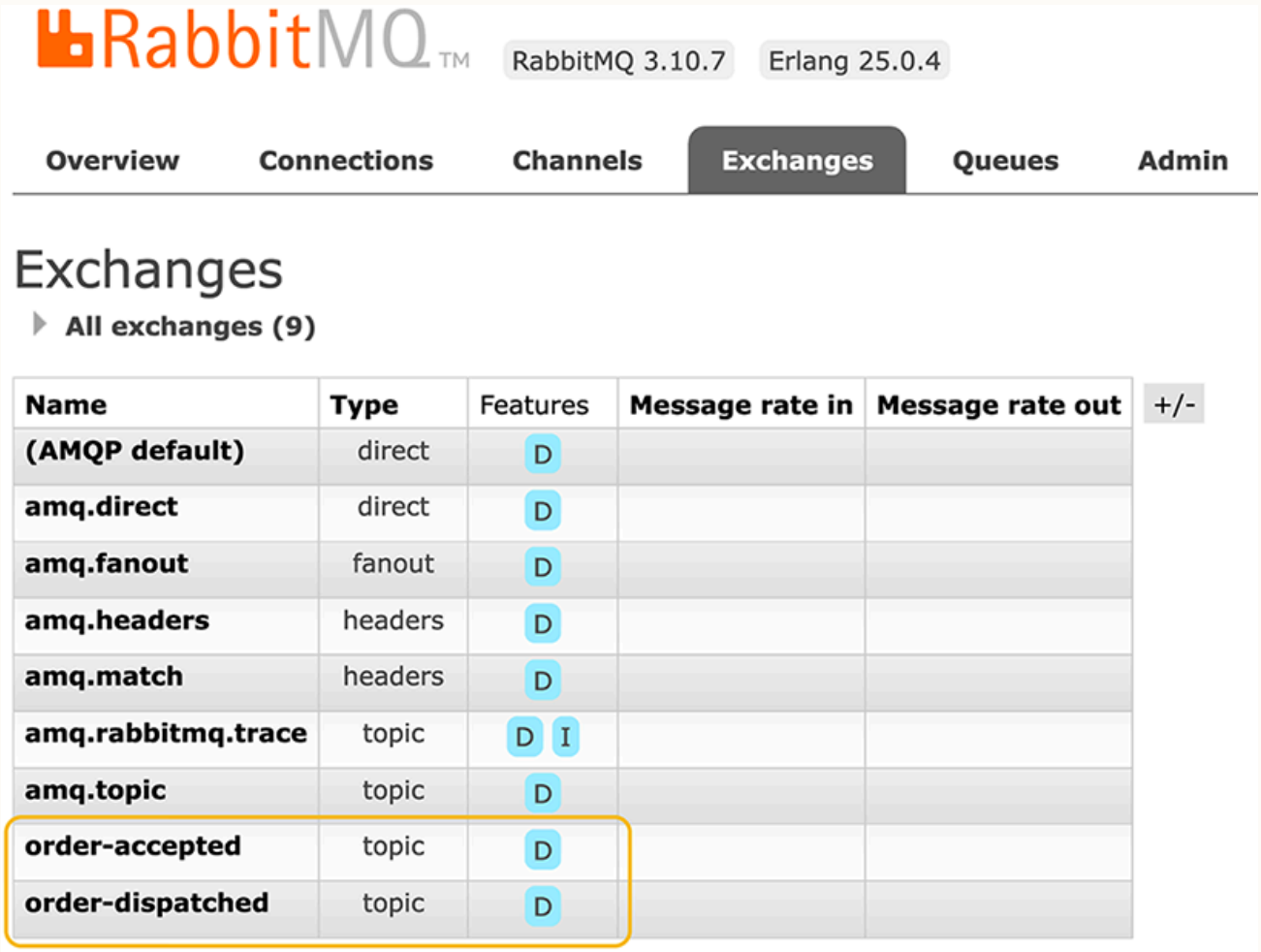


Figure 10.12 Spring Cloud Stream maps the two destination binding to two exchanges in RabbitMQ.

Next, let's take a look at the queues. In Dispatcher Service we configured a packlabel-in-0 binding and a consumer group. That's the only input channel for the application, so it should result in a single queue. Let's verify that. In the RabbitMQ management console, as illustrated in figure 10.13, you can see a durable order-accepted.dispatcher-service queue in the Queues section.

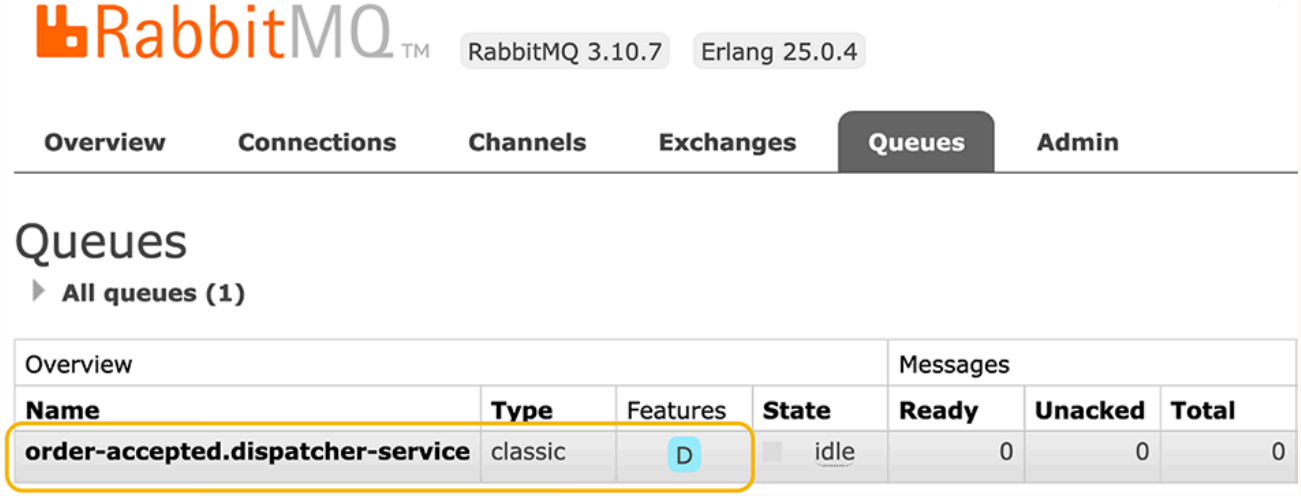


Figure 10.13 Spring Cloud Stream maps each input binding to a queue, named according to the configured consumer group.

NOTE No queue has been created for the packlabel-out-0 binding because no consumer subscribed to it. Later you'll see that a queue will be created after configuring Order Service to listen to it.

We can verify that the integration works by manually sending a message to the order-accepted exchange. If everything is configured correctly, Dispatcher Service will read the message from the order-accepted.dispatcher-service queue, process it through the composed function pack|label, and finally send it to the order-dispatched exchange.

Go to the Exchanges section again, select the order-accepted exchange, and in the Publish Message panel, insert an OrderAcceptedMessage object in JSON format, as shown in figure 10.14. When you're done, click the Publish Message button.

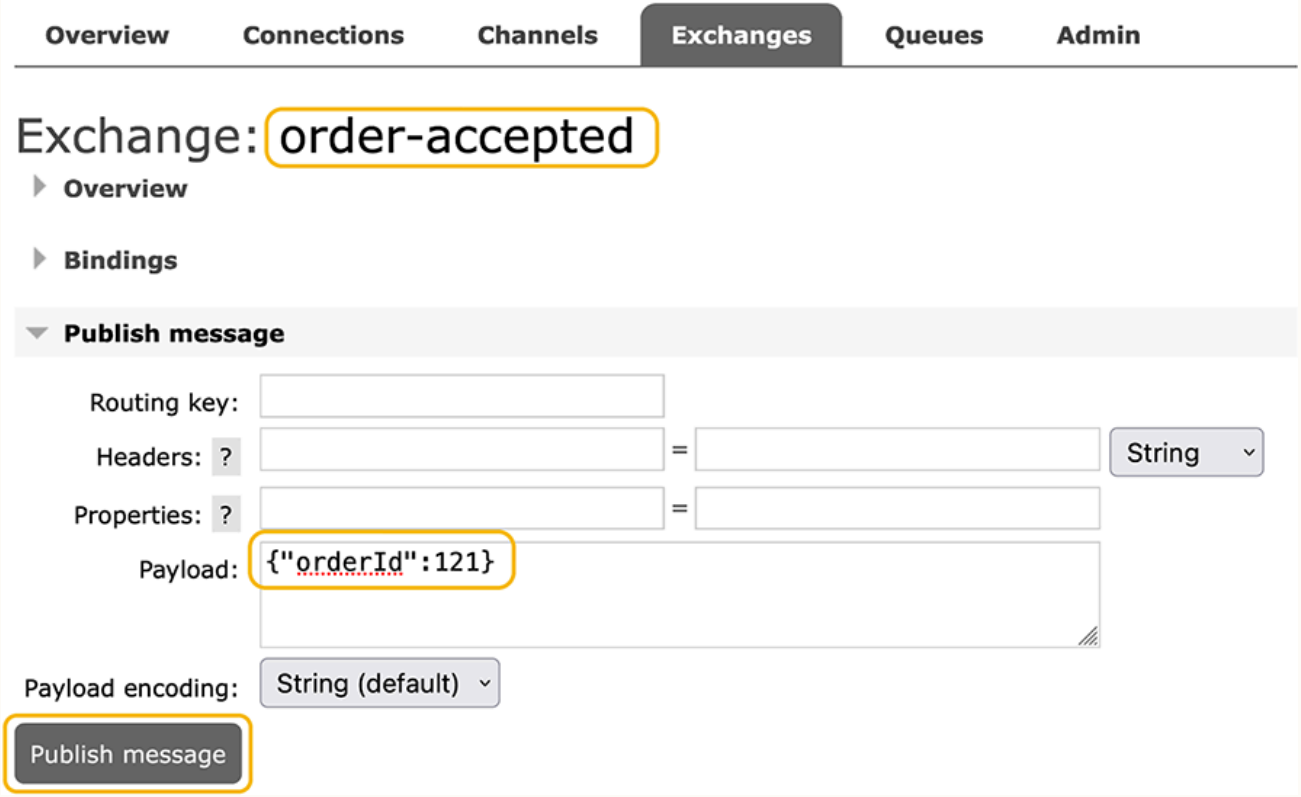


Figure 10.14 You can trigger the data flow in Dispatcher Service by sending a message to the order-accepted exchange.

In the application logs, you should see the following messages signaling that the data flow happened correctly:

```
...c.p.d.DispatchingFunctions: The order with id 394 is packed.
...c.p.d.DispatchingFunctions: The order with id 394 is labeled.
```

The output message has been sent to the order-dispatched exchange, but it has not been routed to any queue because no consumer has subscribed. In the final part of this chapter, we'll complete the flow by defining a supplier in Order Service to publish messages to the order-accepted exchange whenever an order is accepted, and a consumer to read messages from the order-dispatched queue whenever an order is dispatched. But first, let's add some tests to verify the integration with the Spring Cloud Stream binder.

Before moving on, stop the application process with Ctrl-C and the RabbitMQ container with docker-compose down.

10.4.3 Writing integration tests with a test binder

As I have stressed several times, Spring Cloud Function and Spring Cloud Stream's whole philosophy is about keeping the application's business logic infrastructure—and middle-

ware—neutral. After defining the original pack() and label() functions, all we did was update dependencies in Gradle and modify the configuration in application.yml.

It's a good idea to have unit tests covering the business logic, independent of the framework. But it's worth adding a few integration tests to cover the application's behavior in a Spring Cloud Stream context. You should disable the integration tests you wrote earlier in the DispatchingFunctionsIntegrationTests class, since now you'll want to test the integration with the external messaging system.

The framework provides a binder specifically for implementing integration tests focusing on the business logic rather than the middleware. Let's see how it works, using Dispatcher Service as an example.

NOTE The test binder provided by Spring Cloud Stream is meant to verify the correct configuration and integration with a technology-agnostic destination binder. If you want to test the application against a specific broker (in our case, it would be for RabbitMQ), you can rely on Testcontainers, as you learned in the previous chapter. I'll leave that up to you as an exercise.

First, add a dependency to the test binder in the build.gradle file of your Dispatcher Service project. Unlike the other dependencies we've been working on so far, the test binder requires a more elaborate syntax to be included. For more information, refer to the Spring Cloud Stream documentation (**https://spring.io/projects/spring-cloud-stream**). Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 10.15 Adding a dependency for the test binder in Dispatcher Service

```
dependencies {
  ...
  testImplementation("org.springframework.cloud:spring-cloud-stream") {
    artifact {
      name = "spring-cloud-stream"
      extension = "jar"
      type ="test-jar"
      classifier = "test-binder"
    }
  }
}
```

Next, create a new FunctionsStreamIntegrationTests class for testing. The test setup consists of three steps:

1. Import the TestChannelBinderConfiguration class providing configuration for the test binder.
2. Inject an InputDestination bean representing the input binding packlabel-in-0 (by default, since it's the only one).
3. Inject an OutputDestination bean representing the output binding packlabel-out-0 (by default, since it's the only one).

The data flow is based on Message objects (from the org.springframework.messaging package). The framework handles type conversion for you transparently when running the application. However, in this type of test, you need to provide Message objects explicitly. You can use MessageBuilder to create the input message, and use the ObjectMapper utility to perform the type conversion from the binary format used for storing message payloads in a broker.

Listing 10.16 Testing the integration with external messaging systems

```
package com.polarbookshop.dispatcherservice;

import java.io.IOException;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.cloud.stream.binder.test.InputDestination;
import org.springframework.cloud.stream.binder.test.OutputDestination;
import org.springframework.cloud.stream.binder.test.
➥TestChannelBinderConfiguration;
import org.springframework.context.annotation.Import;
import org.springframework.integration.support.MessageBuilder;
import org.springframework.messaging.Message;
import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest
@Import(TestChannelBinderConfiguration.class)                    1
class FunctionsStreamIntegrationTests {

  @Autowired
  private InputDestination input;                                2
```

```
    @Autowired
    private OutputDestination output;                    ❸

    @Autowired
    private ObjectMapper objectMapper;                   ❹

    @Test
    void whenOrderAcceptedThenDispatched() throws IOException {
      long orderId = 121;
      Message<OrderAcceptedMessage> inputMessage = MessageBuilder
        .withPayload(new OrderAcceptedMessage(orderId)).build();
      Message<OrderDispatchedMessage> expectedOutputMessage = MessageBuilder
        .withPayload(new OrderDispatchedMessage(orderId)).build();

      this.input.send(inputMessage);                     ❺
      assertThat(objectMapper.readValue(output.receive().getPayload(),
        OrderDispatchedMessage.class))
        .isEqualTo(expectedOutputMessage.getPayload());  ❻
    }
  }
```

❶ Configures the test binder

❷ Represents the input binding packlabel-in-0

❸ Represents the output binding packlabel-out-0

❹ Uses Jackson to deserialize JSON message payloads to Java objects

❺ Sends a message to the input channel

❻ Receives and asserts a message from the output channel

WARNING If you use IntelliJ IDEA, you might get a warning that InputDestination, OutputDestination, and ObjectMapper cannot be autowired. Don't worry. It's a false positive. You can get rid of the warning by annotating the field with @SuppressWarnings("SpringJavaInjectionPointsAutowiringInspection").

Message brokers like RabbitMQ deal with binary data, so any data flowing through them is mapped to byte[] in Java. The conversion between bytes and DTOs is handled by Spring Cloud Stream transparently. But just like for messages, we need to handle that explicitly in this test scenario when asserting the content of the message received from the output channel.

After writing the integration tests, open a Terminal window, navigate to the Dispatcher Service project's root folder, and run the tests:

```
$ ./gradlew test --tests FunctionsStreamIntegrationTests
```

The next section will go through some points to consider regarding resilient integrations with messaging systems.

10.4.4 Making messaging resilient to failures

Event-driven architectures solve some issues affecting synchronous request/response interactions. For example, if you remove the temporal coupling between applications, you won't need to adopt patterns like circuit breakers, since the communication will be asynchronous. If the consumer is momentarily unavailable while the producer sends a message, it doesn't matter. The consumer will receive the message once it's up and running again.

In software engineering, there are no silver bullets. Everything comes at a cost. On the one hand, applications that are decoupled can operate more independently. On the other hand, you introduced a new component in your system that needs to be deployed and maintained: the message broker.

Assuming that part is taken care of by the platform, there's still something for you to do as the application developer. When an event happens and your application wants to publish a message, something might go wrong. Retries and timeouts are still helpful, but this time we'll use them to make the interaction between application and broker more resilient. Spring Cloud Stream uses the retry pattern with an exponential backoff strategy by default, relying on the Spring Retry library for imperative consumers and the retryWhen() Reactor operator for reactive consumers (the one you learned about in chapter 8). As usual, you can customize it via configuration properties.

Spring Cloud Stream defines several defaults to make the interaction more resilient, including error channels and graceful shutdown. You can configure different aspects of message processing, including dead-letter queues, acknowledgment flows, and republishing messages on error.

RabbitMQ itself has several features in place to improve reliability and resilience. Among other things, it guarantees that each message is delivered at least once. Be aware that consumers in your applications might receive the same message twice, so your business logic should know how to identify and handle duplicates.

I won't go further into the details, since this is an extensive subject that would require several dedicated chapters to cover it adequately. Instead, I encourage you to read the documentation for the different projects involved in your event-driven architecture: RabbitMQ (**https://rabbitmq.com**), Spring AMQP (**https://spring.io/projects/spring-amqp**), and Spring Cloud Stream (**https://spring.io/projects/spring-cloud-stream**). You can also check out the event-driven patterns described in Sam Newman's *Building Microservices* (O'Reilly, 2021) and Chris Richardson's *Microservices Patterns* (Manning, 2018).

In the last part of the chapter, you'll work with suppliers and consumers and complete the order flow for the Polar Bookshop system.

10.5 Producing and consuming messages with Spring Cloud Stream

In previous sections, you learned about the functional programming paradigm and how it fits in the Spring ecosystem, using Spring Cloud Function and Spring Cloud Stream. This last section will guide you through the implementation of producers and consumers.

As you'll see, consumers are not that different from the functions you wrote in Dispatcher Service. On the other hand, producers are slightly different because, unlike functions and consumers, they are not naturally activated. I'll show you how to use them both in Order Service while implementing the last part of the order flow for the Polar Bookshop system.

0.5.1 Implementing event consumers, and the problem of idempotency

The Dispatcher Service application we previously built produces messages when orders are dispatched. The Order Service should be notified when that happens so that it can update the order status in the database.

First, open your Order Service project (order-service), and add the dependencies on Spring Cloud Stream and the test binder in the build.gradle file. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 10.17 Adding dependency for Spring Cloud Stream and test binder

```
dependencies {
  ...
  implementation 'org.springframework.cloud:
  ➥spring-cloud-stream-binder-rabbit'
```

```
  testImplementation("org.springframework.cloud:spring-cloud-stream") {
    artifact {
      name = "spring-cloud-stream"
      extension = "jar"
      type ="test-jar"
      classifier = "test-binder"
    }
  }
}
```

Next we need to model the event we'd like Order Service to listen to. Create a new com.polarbookshop.orderservice.order.event package, and add an OrderDispatchedMessage class to hold the identifier for a dispatched order.

Listing 10.18 A DTO representing the event about orders being dispatched

```
package com.polarbookshop.orderservice.order.event;

public record OrderDispatchedMessage (
  Long orderId
){}
```

Now we'll implement the business logic using a functional approach. Create an OrderFunctions class (com.polarbookshop.orderservice.order.event package), and implement a function to consume the messages produced by the Dispatcher Service application when an order is dispatched. The function will be a Consumer responsible for listening to the incoming messages and updating the database entities accordingly. Consumer objects are functions with input but no output. To keep the function clean and readable, we'll move the processing of OrderDispatchedMessage objects to the OrderService class (which we'll implement in a minute).

Listing 10.19 Consuming messages from RabbitMQ

```
package com.polarbookshop.orderservice.order.event;

import java.util.function.Consumer;
import com.polarbookshop.orderservice.order.domain.OrderService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import reactor.core.publisher.Flux;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OrderFunctions {

  private static final Logger log =
    LoggerFactory.getLogger(OrderFunctions.class);

  @Bean
  public Consumer<Flux<OrderDispatchedMessage>> dispatchOrder(
    OrderService orderService
  ) {
    return flux ->
      orderService.consumeOrderDispatchedEvent(flux)                    (1)
        .doOnNext(order -> log.info("The order with id {} is dispatched",
          order.id()))                                                  (2)
        .subscribe();                                                   (3)
  }
}
```

(1) For each dispatched message, it updates the related order in the database.

(2) For each order updated in the database, it logs a message.

(3) Subscribes to the reactive stream in order to activate it. Without a subscriber, no data flows through the stream.

Order Service is a reactive application, so the dispatchOrder function will consume messages as a reactive stream (a Flux of OrderDispatchedMessage). Reactive streams are *activated* only if there's a subscriber interested in receiving the data. For that reason, it's critical that we end the reactive stream by subscribing to it, or else no data will ever be processed. In previous examples, the subscription part was handled transparently by the framework (for example, when using reactive streams to return data via a REST endpoint or to send data to a backing service). In this case, we have to explicitly do that with the subscribe() clause.

Next, let's implement the consumeOrderDispatchedMessageEvent() method in the OrderService class to update the status in the database for an existing order after it's dispatched.

Listing 10.20 Implementing the logic for updating an order as dispatched

```
@Service
public class OrderService {
 ...

 public Flux<Order> consumeOrderDispatchedEvent(
  Flux<OrderDispatchedMessage> flux
 ) {
  return flux                                    (1)
   .flatMap(message ->
    orderRepository.findById(message.orderId()))  (2)
   .map(this::buildDispatchedOrder)              (3)
   .flatMap(orderRepository::save);              (4)
 }

 private Order buildDispatchedOrder(Order existingOrder) {
  return new Order(                              (5)
   existingOrder.id(),
   existingOrder.bookIsbn(),
   existingOrder.bookName(),
   existingOrder.bookPrice(),
   existingOrder.quantity(),
   OrderStatus.DISPATCHED,
   existingOrder.createdDate(),
   existingOrder.lastModifiedDate(),
   existingOrder.version()
  );
 }
}
```

(1) Accepts a reactive stream of OrderDispatchedMessage objects as input

(2) For each object emitted to the stream, it reads the related order from the database.

(3) Updates the order with the "dispatched" status

(4) Saves the updated order in the database

(5) Given an order, it returns a new record with the "dispatched" status.

Consumers are triggered by a message arriving in the queue. RabbitMQ provides an *at-least-one delivered* guarantee, so you need to be aware of possible duplicates. The code we implemented updates the status of the specific order to be DISPATCHED, an operation that can be executed several times with the same result. Since the operation is

idempotent, the code is resilient to duplicates. A further optimization would be to check for the status and skip the update operation if it's already dispatched.

Finally, we need to configure Spring Cloud Stream in the application.yml file so that the dispatchOrder-in-0 binding (inferred from the dispatchOrder function name) is mapped to the order-dispatched exchange in RabbitMQ. Also, remember to define dispatchOrder as the function that Spring Cloud Function should manage, and the integration with RabbitMQ.

Listing 10.21 Configuring Cloud Stream bindings and RabbitMQ integration

```
spring:
  cloud:
    function:
      definition: dispatchOrder                ❶
    stream:
      bindings:
        dispatchOrder-in-0:                    ❷
          destination: order-dispatched        ❸
          group: {spring.application.name}     ❹
  rabbitmq:                                    ❺
    host: localhost
    port: 5672
    username: user
    password: password
    connection-timeout: 5s
```

❶ Definition of the function managed by Spring Cloud Function

❷ The input binding

❸ The actual name at the broker that the binder binds to (the exchange in RabbitMQ)

❹ The consumer group interested in the destination (the same as the application name)

❺ Configures the integrations with RabbitMQ

As you can see, it works the same way as the functions in Dispatcher Service. The consumers in Order Service will be part of the order-service consumer group, and Spring Cloud Stream will define a message channel between them and an order-dispatched.order-service queue in RabbitMQ.

Next, we'll complete the order flow by defining a supplier responsible for triggering the whole procedure.

10.5.2 Implementing event producers, and the problem of atomicity

Suppliers are message sources. They produce messages when an event happens. In Order Service, a supplier should notify the interested parties (in this case, Dispatcher Service) whenever an order has been accepted. Unlike functions and consumers, suppliers need to be activated. They act only upon invocation.

Spring Cloud Stream provides a few ways to define suppliers and cover different scenarios. In our case, the event source is not a message broker, but a REST endpoint. When a user sends a POST request to Order Service for purchasing a book, we want to publish an event signaling whether the order has been accepted.

Let's start by modeling that event as a DTO. It will be the same as the OrderAcceptedMessage record we used in Dispatcher Service. Add the record to the com.polarbookshop.orderservice.order.event package in your Order Service project (order-service).

Listing 10.22 A DTO representing the event about orders being accepted

```
package com.polarbookshop.orderservice.order.event;

public record OrderAcceptedMessage (
  Long orderId
){}
```

We can bridge the REST layer with the stream part of the application using a StreamBridge object that allows us to send data to a specific destination imperatively. Let's break this new functionality down. First, we can implement a method that accepts an Order object as input, verifies it's accepted, builds an OrderAcceptedMessage object, and sends it to a RabbitMQ destination using StreamBridge.

Open the OrderService class, autowire a StreamBridge object, and define a new publishOrderAcceptedEvent method.

Listing 10.23 Implementing the logic for publishing events to a destination

```
package com.polarbookshop.orderservice.order.domain;

import com.polarbookshop.orderservice.book.BookClient;
import com.polarbookshop.orderservice.order.event.OrderAcceptedMessage;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.stream.function.StreamBridge;
import org.springframework.stereotype.Service;
...

@Service
public class OrderService {
  private static final Logger log =
    LoggerFactory.getLogger(OrderService.class);

  private final BookClient bookClient;
  private final OrderRepository orderRepository;
  private final StreamBridge streamBridge;

  public OrderService(BookClient bookClient,
    StreamBridge streamBridge, OrderRepository orderRepository
  ) {
    this.bookClient = bookClient;
    this.orderRepository = orderRepository;
    this.streamBridge = streamBridge;
  }

  ...

  private void publishOrderAcceptedEvent(Order order) {
    if (!order.status().equals(OrderStatus.ACCEPTED)) {
      return;                                    (1)
    }
    var orderAcceptedMessage =
      new OrderAcceptedMessage(order.id());          (2)
    log.info("Sending order accepted event with id: {}", order.id());
    var result = streamBridge.send("acceptOrder-out-0",
      orderAcceptedMessage);                         (3)
    log.info("Result of sending data for order with id {}: {}",
      order.id(), result);
  }
}
```

(1) If the order is not accepted, it does nothing.

❷ Builds a message to notify that an order has been accepted

❸ Explicitly sends a message to the acceptOrder-out-0 binding

Since the data source is a REST endpoint, there is no Supplier bean we can register with Spring Cloud Function, and therefore there is no trigger for the framework to create the necessary bindings with RabbitMQ. Yet, in listing 10.23, StreamBridge is used to send data to an acceptOrder-out-0 binding. Where does it come from? There is no acceptOrder function!

At startup time, Spring Cloud Stream will notice that StreamBridge wants to publish messages via an acceptOrder-out-0 binding, and it will create one automatically. Similar to the bindings created from functions, we can configure the destination name in RabbitMQ. Open the application.yml file and configure the binding as follows.

Listing 10.24 Configuring Cloud Stream output binding

```
spring:
  cloud:
    function:
      definition: dispatchOrder
    stream:
      bindings:
        dispatchOrder-in-0:
          destination: order-dispatched
          group: ${spring.application.name}
        acceptOrder-out-0:                    ❶
          destination: order-accepted         ❷
```

❶ Output binding created and managed by StreamBridge

❷ The actual name at the broker that the binder binds to (the exchange in RabbitMQ)

All that's left now is calling the method whenever a submitted order is accepted. That's a critical point and one of the aspects characterizing the *saga pattern*, a popular alternative to distributed transactions in microservice architectures. To ensure consistency in your system, persisting an order in the database and sending a message about it must be done atomically. Either both operations succeed, or they both must fail. A simple yet effective way to ensure atomicity is by wrapping the two operations in a local transaction. To do that, we can rely on the built-in Spring transaction management functionality.

NOTE The saga pattern is described extensively in chapter 4 of Chris Richardson's book, *Microservices Patterns* (Manning, 2018; https://livebook.manning.com/book/microservices-patterns/chapter-4). I recommend you check it out if you're interested in designing business transactions that span multiple applications.

In the OrderService class, modify the submitOrder() method to call the publishOrderAcceptedEvent method, and annotate it with @Transactional.

Listing 10.25 Defining a saga transaction with database and event broker

```
@Service
public class OrderService {
  ...

  @Transactional                                    (1)
  public Mono<Order> submitOrder(String isbn, int quantity) {
    return bookClient.getBookByIsbn(isbn)
      .map(book -> buildAcceptedOrder(book, quantity))
      .defaultIfEmpty(buildRejectedOrder(isbn, quantity))
      .flatMap(orderRepository::save)               (2)
      .doOnNext(this::publishOrderAcceptedEvent);   (3)
  }

  private void publishOrderAcceptedEvent(Order order) {
    if (!order.status().equals(OrderStatus.ACCEPTED)) {
      return;
    }
    var orderAcceptedMessage = new OrderAcceptedMessage(order.id());
    log.info("Sending order accepted event with id: {}", order.id());
    var result = streamBridge.send("acceptOrder-out-0",
      orderAcceptedMessage);
    log.info("Result of sending data for order with id {}: {}",
      order.id(), result);
  }
}
```

(1) Executes the method in a local transaction

(2) Saves the order in the database

(3) Publishes an event if the order is accepted

Spring Boot comes preconfigured with transaction management functionality and can handle transactional operations involving relational databases (as you learned in chapter 5). However, the channel established with RabbitMQ for the message producer is not transactional by default. To make the event-publishing operation join the existing transaction, we need to enable RabbitMQ's transactional support for the message producer in the application.yml file.

Listing 10.26 Configuring the output binding to be transactional

```
spring:
  cloud:
    function:
      definition: dispatchOrder
    stream:
      bindings:
        dispatchOrder-in-0:
          destination: order-dispatched
          group: ${spring.application.name}
        acceptOrder-out-0:
          destination: order-accepted
      rabbit:                          (1)
        bindings:
          acceptOrder-out-0:
            producer:
              transacted: true         (2)
```

(1) RabbitMQ-specific configuration for the Spring Cloud Stream bindings

(2) Makes the acceptOrder-out-0 binding transactional

Now you can write new integration tests for the supplier and the consumer, like we did for the functions in Dispatcher Service. I'll leave the autotests to you, since you have the necessary tools now. If you need inspiration, check out the source code accompanying this book (Chapter10/10-end/order-service).

You will also need to import the configuration for the test binder (`@Import(TestChannelBinderConfiguration.class)`) in the existing `OrderServiceApplicationTests` class to make it work.

We've had a nice journey through event-driven models, functions, and messaging systems. Before wrapping up, let's look at the order flow in action. First, start RabbitMQ, PostgreSQL (`docker-compose up -d polar-rabbitmq polar-postgres`), and

Dispatcher Service (`./gradlew bootRun`). Then run Catalog Service and Order Service (`./gradlew bootRun` or from Docker Compose after building the images first).

Once all those services are up and running, add a new book to the catalog:

```
$ http POST :9001/books author="Jon Snow" \
    title="All I don't know about the Arctic" isbn="1234567897" \
    price=9.90 publisher="Polarsophia"
```

Then order three copies of that book:

```
$ http POST :9002/orders isbn=1234567897 quantity=3
```

If you order a book that exists, the order will be accepted, and Order Service will publish an `OrderAcceptedEvent` message. Dispatcher Service, subscribed to that same event, will process the order and publish an `OrderDispatchedEvent` message. Order Service will be notified and update the order status in the database.

TIP You can follow the message flow by checking the application logs from Order Service and Dispatcher Service.

Now the moment of the truth. Fetch the order from Order Service:

```
$ http :9002/orders
```

The status should be `DISPATCHED`:


```
{
  "bookIsbn": "1234567897",
  "bookName": "All I don't know about the Arctic - Jon Snow",
  "bookPrice": 9.9,
  "createdDate": "2022-06-06T19:40:33.426610Z",
  "id": 1,
  "lastModifiedDate": "2022-06-06T19:40:33.866588Z",
  "quantity": 3,
  "status": "DISPATCHED",
  "version": 2
}
```

And it is. Great job! When you're done testing the system, stop all the applications (Ctrl-C) and Docker containers (`docker-compose down`).

That concludes the main implementation of the business logic for the Polar Bookshop system. The next chapter will cover security for cloud native applications using Spring Security, OAuth 2.1, and OpenID Connect.

Polar Labs

Feel free to apply what you've learned in the previous chapters and prepare the Dispatcher Service application for deployment.

1. Add Spring Cloud Config Client to Dispatcher Service to make it fetch configuration data from Config Service.
2. Configure the Cloud Native Buildpacks integration, containerize the application, and define the commit stage of the deployment pipeline.
3. Write the Deployment and Service manifests for deploying Dispatcher Service to a Kubernetes cluster.
4. Configure Tilt to automate the Dispatcher Service's deployment to your local Kubernetes cluster initialized with minikube.

Then update the Docker Compose specification and the Kubernetes manifests to configure the RabbitMQ integration for Order Service.

You can refer to the Chapter10/10-end folder in the code repository accompanying the book to check on the final result (**https://github.com/ThomasVitale/cloud-native-spring-in-action**). Deploy the backing services from the manifests, available in the Chapter10/10-end/polar-deployment/kubernetes/platform/development folder, with `kubectl apply -f services`.

Summary

- Event-driven architectures are distributed systems that interact with each other by producing and consuming events.
- An event is something relevant that happened in a system.
- In the pub/sub model, producers publish events, which are sent to all subscribers to be consumed.
- Event processing platforms like RabbitMQ and Kafka are responsible for collecting events from the producers, routing, and distributing them to the interested consumers.

- In the AMQP protocol, producers send messages to an exchange in a broker that forwards them to queues according to specific routing algorithms.
- In the AMQP protocol, consumers receive messages from the queues in the broker.
- In the AMQP protocol, messages are data structures composed of key/value attributes and a binary payload.
- RabbitMQ is a message broker based on the AMQP protocol that you can use to implement event-driven architectures based on the pub/sub model.
- RabbitMQ provides high availability, resilience, and data replication.
- Spring Cloud Function enables you to implement your business logic using the standard Java `Function`, `Supplier`, and `Consumer` interfaces.
- Spring Cloud Function wraps your function and provides several exciting features like transparent type conversion and function composition.
- Functions implemented in the context of Spring Cloud Function can be exposed and integrated with external systems in different ways.
- Functions can be exposed as REST endpoints, packaged, and deployed in a FaaS platform as serverless applications (Knative, AWS Lambda, Azure Function, Google Cloud Functions), or they can be bound to message channels.
- Spring Cloud Stream, built on top of Spring Cloud Function, provides you with all the necessary plumbing to integrate your functions with external messaging systems like RabbitMQ or Kafka.
- Once you implement your functions, you don't have to make any changes to your code. You only need to add a dependency on Spring Cloud Stream and configure it to adapt to your needs.
- In Spring Cloud Stream, destination binders provide integration with external messaging systems.
- In Spring Cloud Stream, destination bindings (input and output) bridge the producers and consumers in your applications with exchanges and queues in a message broker like RabbitMQ.
- Functions and consumers are activated automatically when new messages arrive.
- Suppliers need to be explicitly activated, such as by explicitly sending a message to a destination binding.

---

1. See Sam Newman, *Monolith to Microservices* (O'Reilly, 2019).

‹ 9 API gateway and circuit breakers

11 Security: Authentication and SPA ›